EXPERIMENT - 3

Aim : Create a Cryptocurrency using Python and perform mining in the Blockchain created.

Lab Outcomes (LO) : LO4: Demonstrate the concept of Blockchain in real-world applications.
After completing this experiment, the student will be able to:

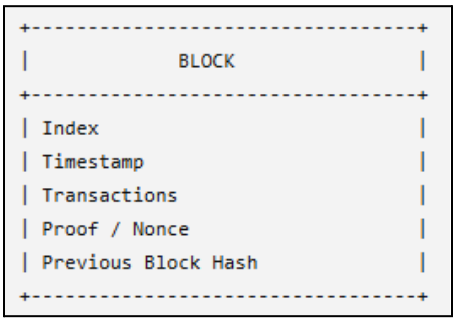
1. Understand the working of a cryptocurrency using Blockchain.
2. Create and manage multiple Blockchain nodes.
3. Perform transactions between users.
4. Mine blocks using Proof-of-Work.
5. Synchronize Blockchain data among multiple nodes.
6. Understand the role of consensus in a decentralized network.

Theory

1. Blockchain Overview

Blockchain is a **decentralized and distributed digital ledger** that stores data in the form of connected blocks. Each block contains transactions, timestamp, previous block hash and proof.

The blocks are linked using cryptographic hashes, which makes the blockchain secure and difficult to modify.



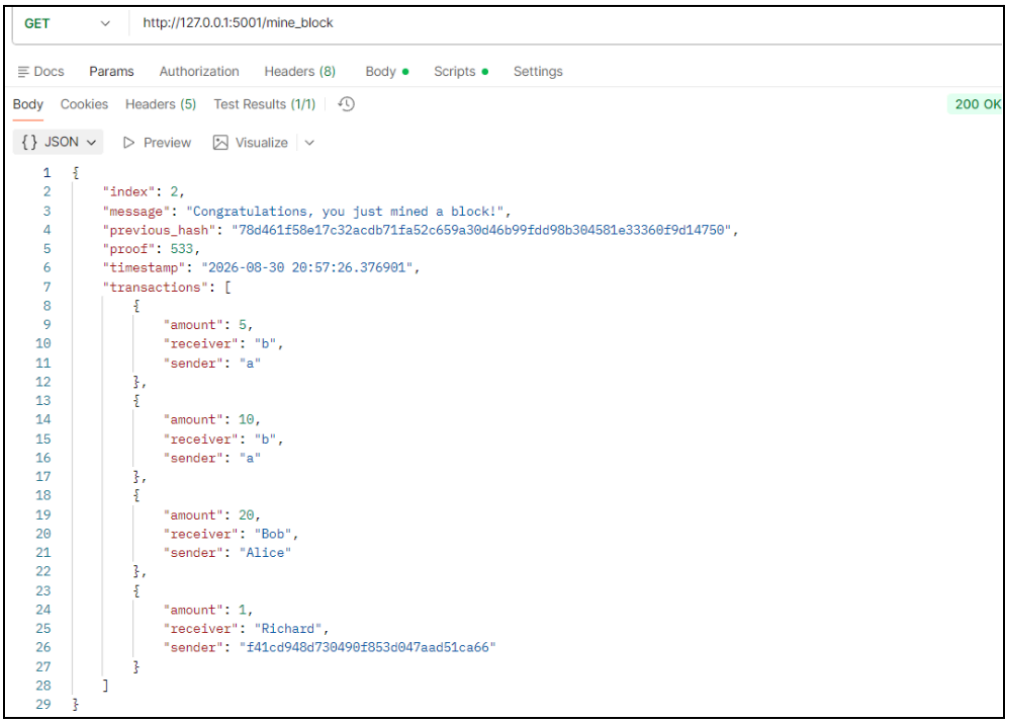
2. Mining

Mining is the process of adding a new block to the Blockchain. The miner collects pending transactions and solves a **Proof-of-Work** problem by finding a valid proof.

After successful mining, the new block is added to the chain and the miner receives a cryptocurrency reward.

Mining process:

1. Collect pending transactions.
2. Find a valid proof using Proof-of-Work.
3. Create a new block.
4. Add the block to the Blockchain.
5. Synchronize the block with other nodes.

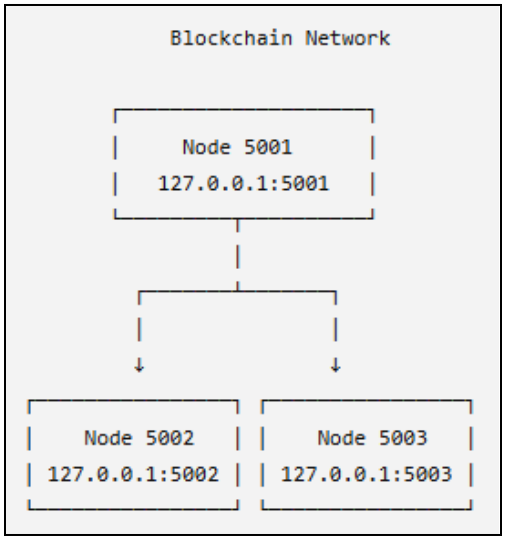


mine_block

3. Multi-Node Blockchain Network

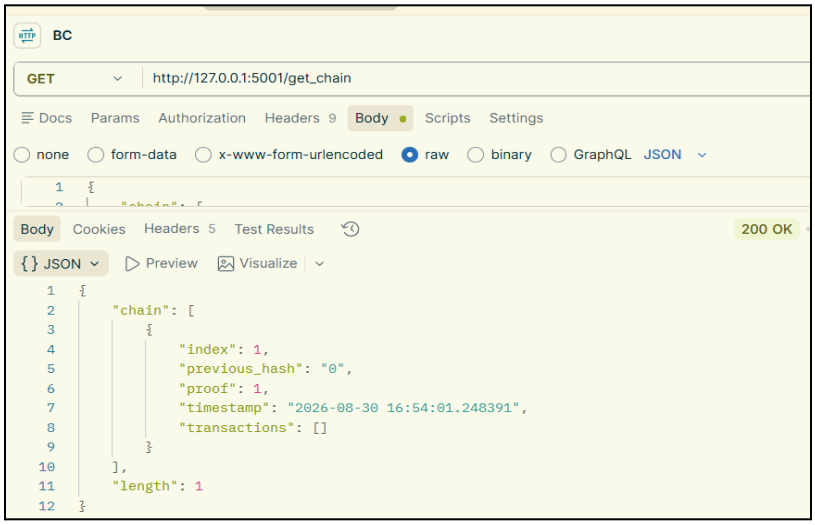
In this experiment, three Blockchain nodes are created using different ports:

- Node 1 → `5001`
- Node 2 → `5002`
- Node 3 → `5003`

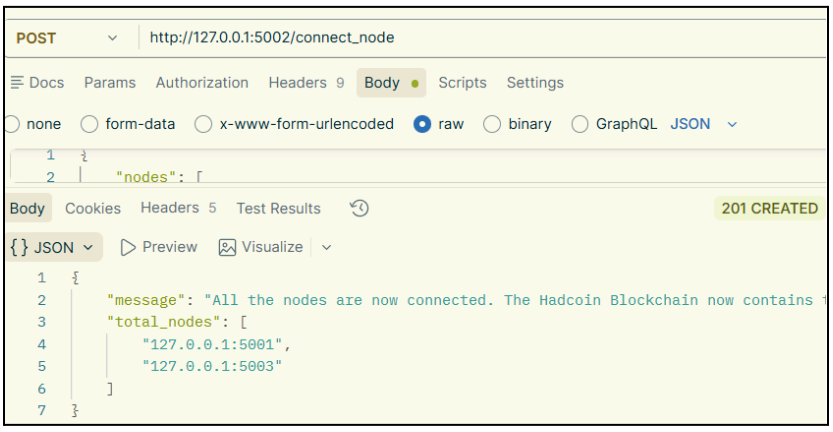


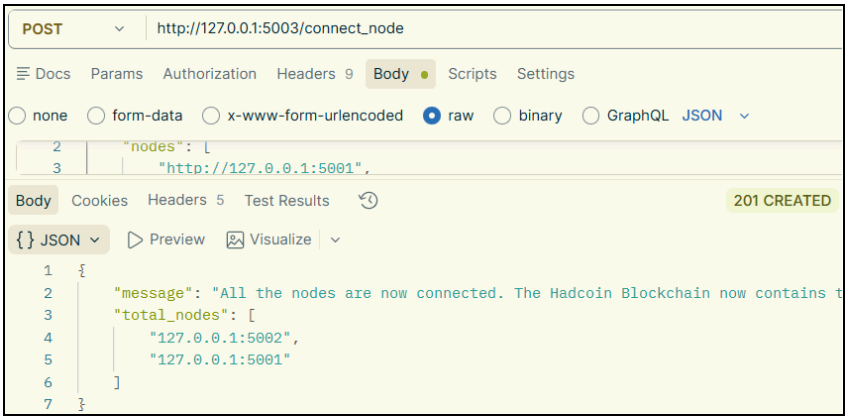
Each node maintains its own copy of the Blockchain and communicates with other nodes through HTTP requests.

Connections :









4. Transactions

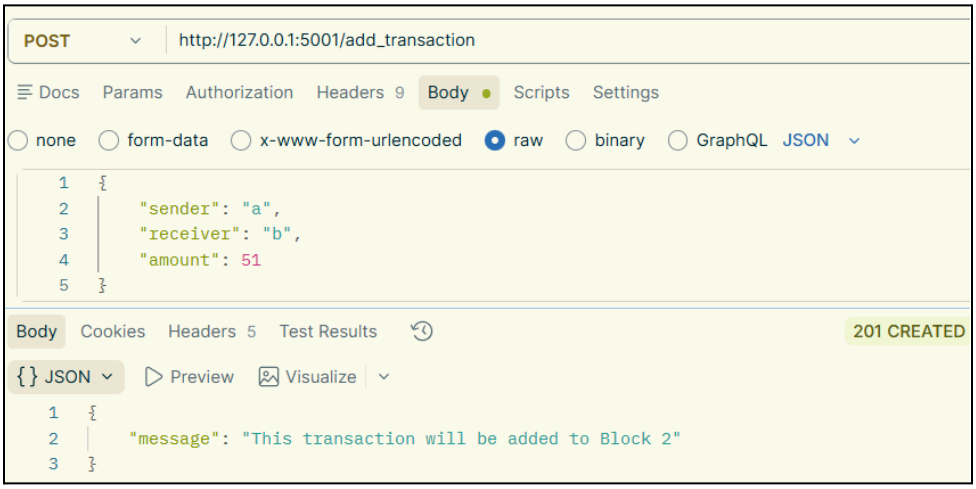
A transaction represents the transfer of cryptocurrency from one user to another. It contains:

- **Sender**
- **Receiver**
- **Amount**

For example:

```
{
  "sender": "a",
  "receiver": "b",
  "amount": 52
}
```

Transactions are initially kept as **pending transactions** and are included in a block when mining is performed.

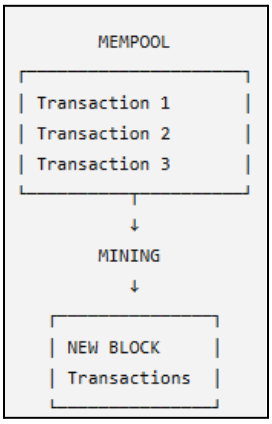


5. Mempool

A **mempool (memory pool)** stores valid transactions that have been submitted to the network but have not yet been included in a block.

When a miner starts mining, it takes the pending transactions from the mempool and adds them to the new block.

After the block is successfully mined, the transactions are removed from the pending transaction list.



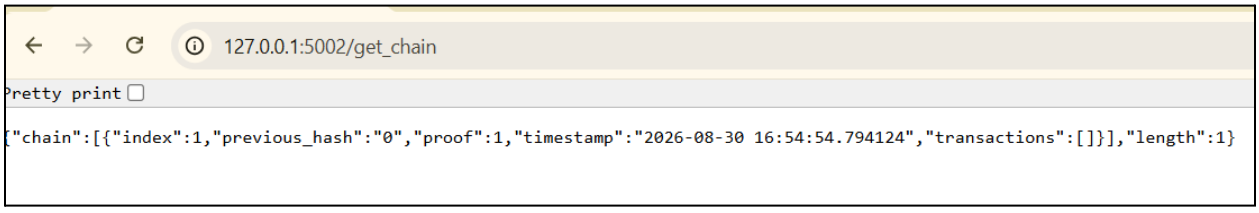
6. Consensus and Longest Chain Rule

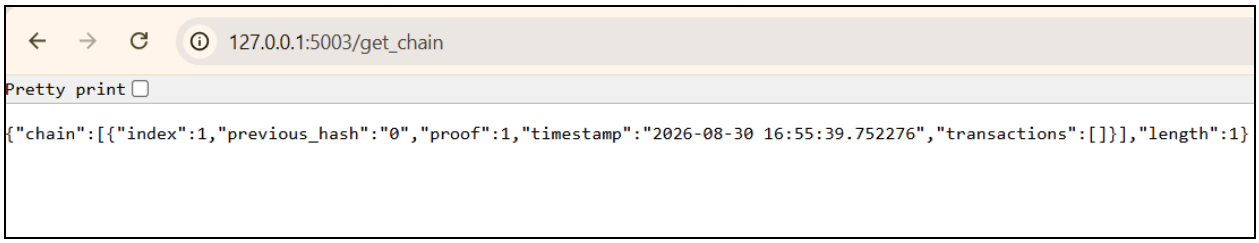
When different nodes have different versions of the Blockchain, a consensus mechanism is required to decide which chain should be accepted.

In this experiment, the **Longest Chain Rule** is used. A node replaces its current chain if another node has a longer valid chain.

This helps the three nodes maintain a common Blockchain.



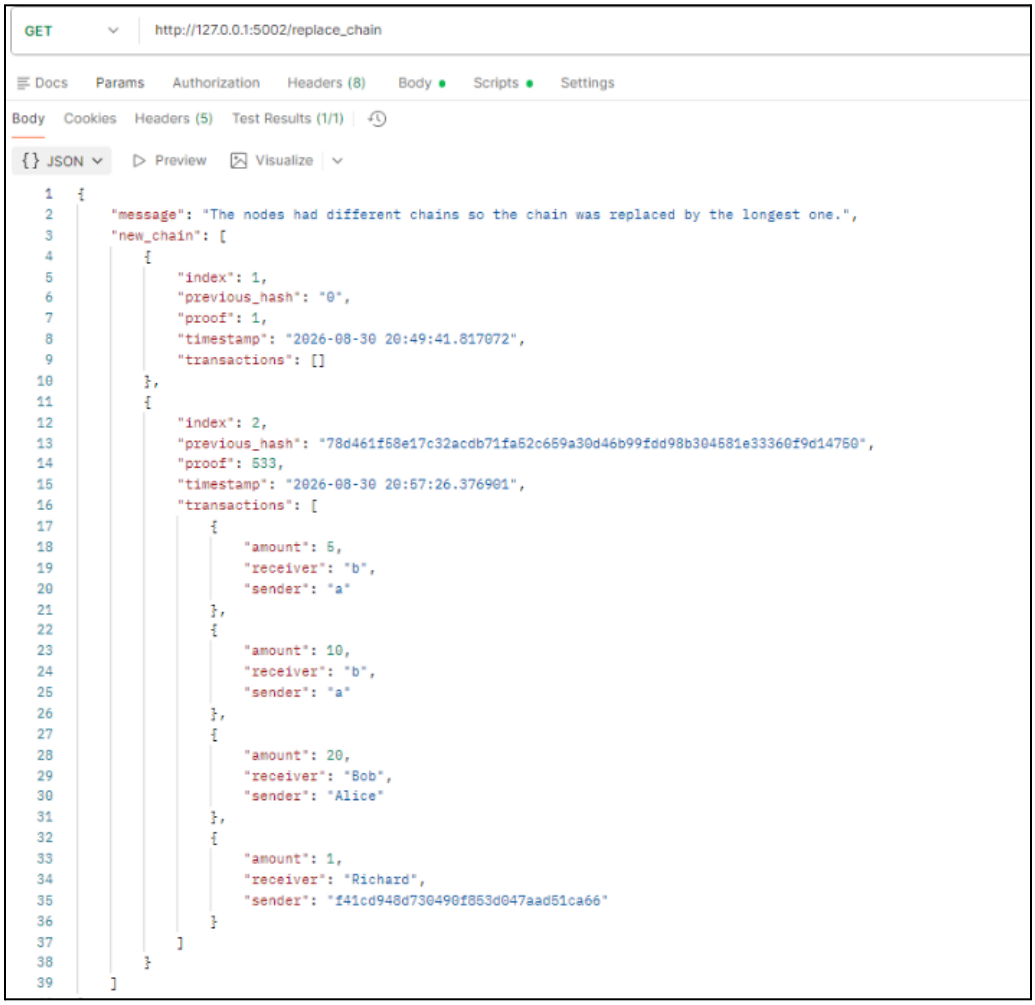




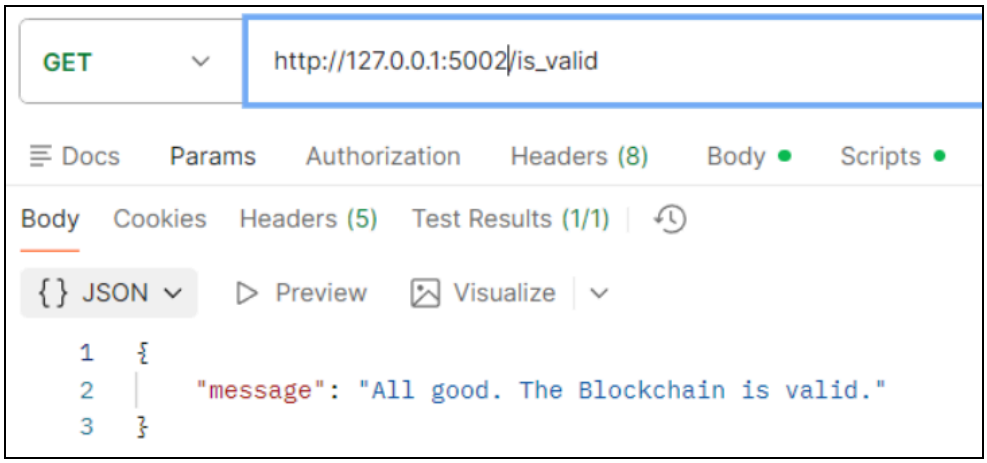
7. Chain Replacement

The `/replace_chain` function checks the Blockchain of connected nodes. If a longer valid chain is found, the current node replaces its chain with the longer one.

This allows the Blockchain network to synchronize different nodes.



replace _chain



is_valid

Challenges in P2P Networks

A **Peer-to-Peer (P2P) network** allows nodes to communicate directly without depending on a central server. Some common challenges are:

1. **Network failures** – Nodes may disconnect or become unavailable.
2. **Synchronization** – All nodes must maintain a consistent copy of the Blockchain.
3. **Security** – Malicious or invalid transactions must be prevented.
4. **Latency** – Transactions and blocks may take time to reach other nodes.
5. **Consensus** – Nodes need a mechanism to agree on the valid Blockchain.

How Transactions are Performed

The transaction process is:
User → Transaction → Pending Transactions/Mempool → Mining → Block → Blockchain
A user submits a transaction using the /add_transaction API. The transaction is stored as pending data. When a miner executes /mine_block, the pending transactions are added to a new block along with the mining reward. The block is then added to the Blockchain.

Libraries and Tools Used

Library / Tool	Purpose
Python	Programming language used for Blockchain implementation
Flask	Creates APIs and runs Blockchain nodes
Requests	Sends HTTP requests between Blockchain nodes

Hashlib	Generates SHA-256 cryptographic hashes
Datetime	Stores block timestamp
UUID	Generates a unique identifier for the miner
URLParse	Extracts network/URL information
Request	Receives data from API requests
JSONify	Converts Python data into JSON responses
Postman	Used to test Blockchain APIs

APIs Used in the Experiment

API	Method	Purpose
/connect_node	POST	Connects Blockchain nodes
/add_transaction	POST	Adds a new transaction
/mine_block	GET	Mines and adds a new block
/get_chain	GET	Displays the Blockchain
/replace_chain	GET	Replaces the chain with a longer valid chain

Procedure

1. Download the three Python files: hadcoin_node_5001.py, hadcoin_node_5002.py and hadcoin_node_5003.py.
2. Create and activate a Python virtual environment.
3. Install Flask and Requests.
4. Run all three nodes in separate terminals.
5. Connect the nodes using /connect_node.
6. Add transactions using /add_transaction.
7. Mine the transactions using /mine_block.
8. View the Blockchain using /get_chain.
9. Use /replace_chain to synchronize the nodes.
10. Modify the program so that transactions are removed from the pending list after being added to a block.

Conclusion

Thus, a simple Cryptocurrency and Blockchain network was implemented using Python and Flask. Three nodes were connected through a P2P network, transactions were added and mined into blocks using Proof-of-Work. The Longest Chain Rule was used to synchronize the Blockchain between nodes. This experiment helped in understanding transactions, mining, mempool, consensus and Blockchain synchronization.